CSE 4101: Artificial Intelligence

Lecture Slide 3: Solving Problems by Searching

Prepared By

Mr. Palash Roy

Lecturer

Department of Computer Science & Engineering

University of Dhaka

What We'll Cover Today

3.1 Problem-Solving Agents — goal formulation, problem formulation, search, execution

3.2 Example Problems — vacuum world, 8-puzzle, and real-world problems

3.3 Search Algorithms — the search tree, frontier, and best-first search

3.4 Uninformed Search Strategies — BFS, Uniform-Cost, DFS, Depth-Limited, Iterative Deepening, Bidirectional

Problem-Solving Agents

Why Do Intelligent Agents Need to Search?

- When the right action isn't immediately obvious, an agent must **plan ahead** — considering a sequence of actions that leads to a goal.
- This computational process is called **search**, and an agent that does this is a **problem-solving agent**.
- Example — a trip to the beach:
  - **Initial plan:** drive to the beach with specific stops and activities.
  - **Unexpected change:** the planned restaurant is closed, and a shortcut road is blocked.
  - **Searching finds alternatives** — a new restaurant, a different route — and the plan adapts.
- Plans are dynamic: **AI systems use search algorithms to continuously build and refine plans as new information arrives.**

The Four-Phase Problem-Solving Process

1. Goal Formulation

The agent decides what it wants to achieve — the desired outcome. This sets the direction for everything that follows.

2. Problem Formulation

The agent translates its goal into a well-defined problem: the initial state, the possible actions, and the criteria for success.

3. Search

The agent simulates sequences of actions in its model before acting in the real world, until it finds a sequence — a solution — that reaches the goal.

4. Execution

The agent carries out the solution's actions, one at a time, in the real world (or a simulation).

Running Example: Getting from Arad to Bucharest

- Our agent is in **Arad** with a non-refundable flight out of **Bucharest** tomorrow.
- It knows the road map (below) — with this information, it can search for a route before driving anywhere.
- This map is the state-space graph: cities are states, roads are actions (each road = two directed actions), and numbers are action costs (distances in miles).

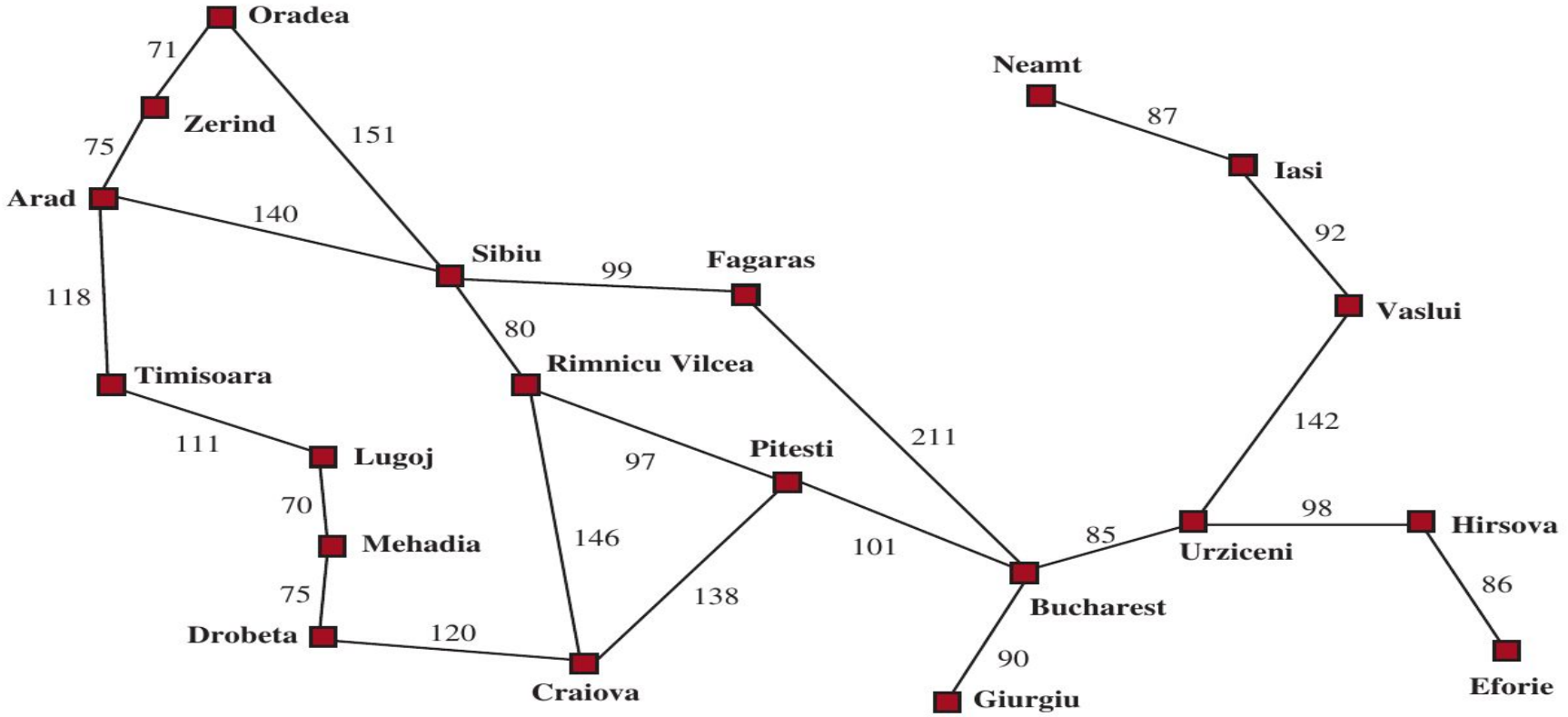


Figure 3.1 A simplified road map of part of Romania, with road distances in miles.

Open-Loop vs. Closed-Loop Execution

- **Open-loop system:** in a fully observable, deterministic, known environment, the solution is a fixed sequence of actions.
  - If the model is correct, the agent can ignore its percepts while executing — 'closing its eyes' — because the plan is guaranteed to reach the goal.
  - Example: drive to Sibiu, then Fagaras, then Bucharest — no need to keep checking.
- **Closed-loop system:** if the environment might be nondeterministic or the model might be wrong, the agent should monitor its percepts as it acts.
  - In partially observable or nondeterministic environments, a solution is a branching strategy — different future actions depending on what is actually perceived.
  - Example: the agent plans to drive from Arad to Sibiu, but needs a contingency in case it ends up in Zerind by accident, or finds a 'Road Closed' sign.

Formal Definition of a Search Problem

- **State space:** the set of all possible states the environment can be in.
- **Initial state:** where the agent starts (e.g., Arad).
- **Goal states:** one or more states that satisfy the agent's objective (e.g., Bucharest).
- **Actions:** ACTIONS(s) returns the set of actions applicable in state s.
- **Transition model:** RESULT(s, a) returns the state that results from doing action a in state s.
- **Action cost function:** ACTION-COST(s, a, s′) gives the numeric cost of an action — reflecting the agent's own performance measure.

Worked notation (Arad):

```
ACTIONS(Arad) =
{ToSibiu, ToTimisoara, ToZerind}


RESULT(Arad, ToZerind) = Zerind
```

Path & Solution:

A path is a sequence of actions. A solution is a path from the initial state to a goal state. An optimal solution has the lowest path cost among all solutions.

Abstraction: Choosing the Right Level of Detail

- **Abstraction** is the process of removing detail from a representation, keeping only what's relevant to the problem.
- Our Romania model ignores traveling companions, the radio, the scenery, traffic, weather — none of it matters for finding a route.
- **Too much detail hurts:** if actions were 'move the right foot forward one centimeter,' the agent would never get out of the parking lot, let alone reach Bucharest.
- A good abstraction is:
  - **Valid** — every abstract solution can be elaborated into a solution in the more detailed world.
  - **Useful** — each abstract action is easier to carry out than the original problem (e.g., 'drive from Arad to Sibiu' needs no further planning by an average driver).
- Without good abstractions, intelligent agents would be completely swamped by the real world.

Example Problems

Standardized Problem: The Vacuum World

- **States:** which cells hold the agent and which hold dirt. Two cells ⟶ 2·2·2 = 8 states.
- **Actions:** Left, Right, Suck.
- **Transition model:** Suck removes dirt from the current cell; Left/Right move the agent (no effect at a wall).
- **Goal state:** every cell is clean.
- **Action cost:** 1 per action.

Two-cell vacuum world:
8 states, 3 actions each
(L = Left, R = Right, S = Suck)

```
  A  B
[Agent|Dirt]  [Dirt]
```

In general, an n-cell vacuum world has

$n \cdot 2^n$

states — this grows fast!

Standardized Problem: The 8-Puzzle

- **States:** the location of each of the 8 numbered tiles + the blank.
- **Actions:** think of the blank moving **Left, Right, Up, or Down** (not all applicable at edges/corners).
- **Transition model:** applying Left to the start state swaps the blank with the tile to its left.
- **Goal state:** typically, the numbers in order (shown right).
- **Action cost:** 1 per move.
- **Even here, abstraction is at work:** we ignore the intermediate sliding motion and any 'shake the board' tricks — only beginning and final states matter.

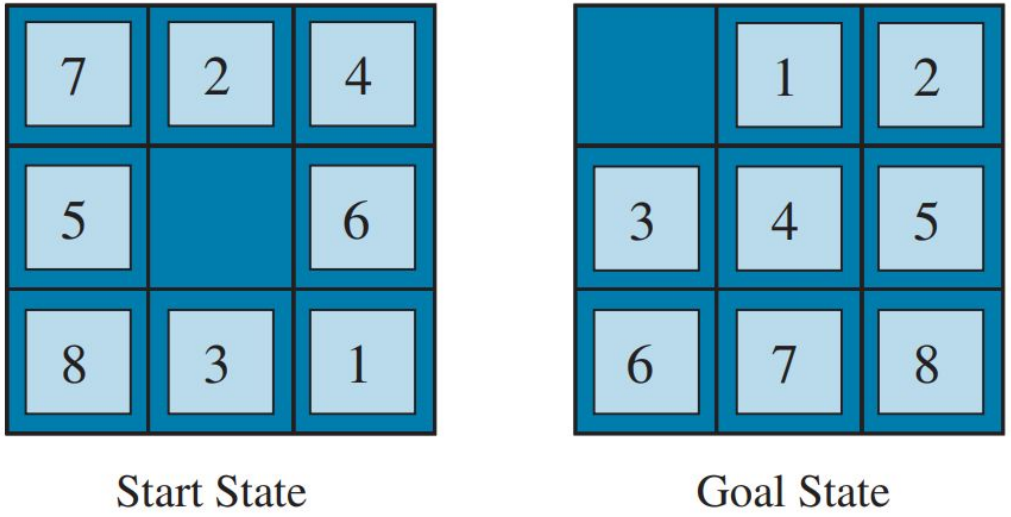


Figure 3.3 A typical instance of the 8-puzzle: start state (left) and goal state (right).

Real-World Problems

Route Finding

Used by GPS apps, airlines, and in-car navigation to find paths through road, rail, or flight networks.

Touring / TSP

Visit a set of cities exactly once and return to the start, minimizing total distance — the Traveling Salesperson Problem.

VLSI Layout

Place millions of components and connections on a chip to minimize area, delay, and stray capacitance.

Robot Navigation

A generalization of route-finding where the robot moves through a continuous, often unmapped, physical space.

Automatic Assembly

Find an order to assemble a complex object's parts so that no step is blocked by an earlier one.

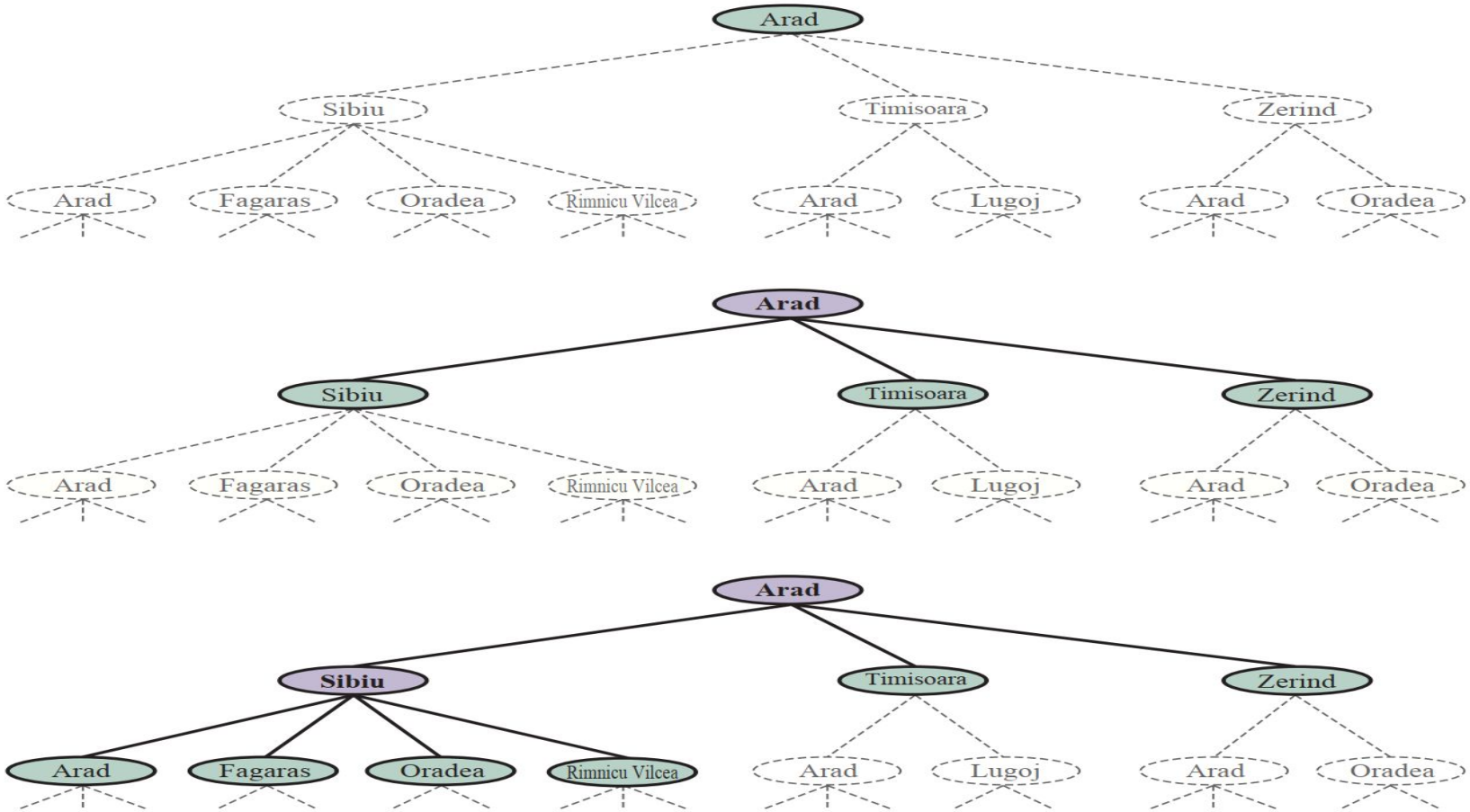
Protein/Gene Design

Search sequences of amino acids or genetic edits to achieve a target 3D structure or function.

Search Algorithms

The Search Tree: Expanding Nodes from the Root

- A search algorithm superimposes a **search tree** over the state-space graph. Each node = a state; each edge = an action.
- The root is the initial state (Arad). To **expand** a node means generating all its child (successor) nodes via ACTIONS and RESULT.
- **The frontier** is the set of generated-but-not-yet-expanded nodes. A state that has had a node generated for it has been **reached**.



Best-First Search: The General Framework

- How do we decide which frontier node to expand next?
- **Best-first search** picks the node n with the minimum value of an **evaluation function f(n)**.
- Every iteration: pop the best node from the frontier; if it's a goal, return it; otherwise expand it and add its children to the frontier.
- Different choices of f(n) give us all the algorithms in this chapter:
  - f(n) = depth of n → Breadth-First Search
  - f(n) = path cost g(n) → Uniform-Cost Search (Dijkstra's algorithm)
  - f(n) = −depth of n → Depth-First Search
- **This unifying view means BFS, UCS, and DFS are really the same algorithm** with a different priority rule.

Uninformed Search Strategies

Uninformed Search: Looking Blindly for Solutions

- An **uninformed** (blind) search algorithm has no clue how close a state is to the goal — it only knows the problem definition: states, actions, and goal test.
- In contrast, an **informed** (heuristic) search — covered in Next Lecture Slide — can estimate distance to the goal (e.g., using straight-line distance).
- We compare uninformed strategies using four criteria:
  - **Completeness** — is it guaranteed to find a solution if one exists?
  - **Optimality** — does it find the lowest-cost solution?
  - **Time complexity** — how long does it take?
  - **Space complexity** — how much memory does it need?

Breadth-First Search (BFS)

- When all actions cost the same, BFS is the natural strategy: expand the **root first**, then all its successors, then their successors, and so on — level by level.
- Implemented using a **FIFO queue** (First-In-First-Out): new (deeper) nodes go to the back; old (shallower) nodes are expanded first.
- Uses an **early goal test** — check for the goal as soon as a node is generated, not when it's popped — for efficiency.
- **Guarantee:** BFS always finds a solution with the minimum number of actions, because it exhausts depth $d-1$ before touching depth d.

BFS Simulation: A Simple Binary Tree (Book Example)

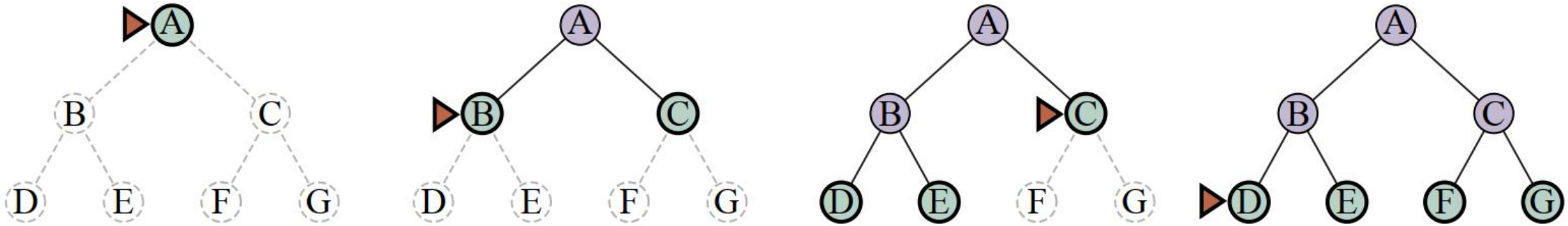
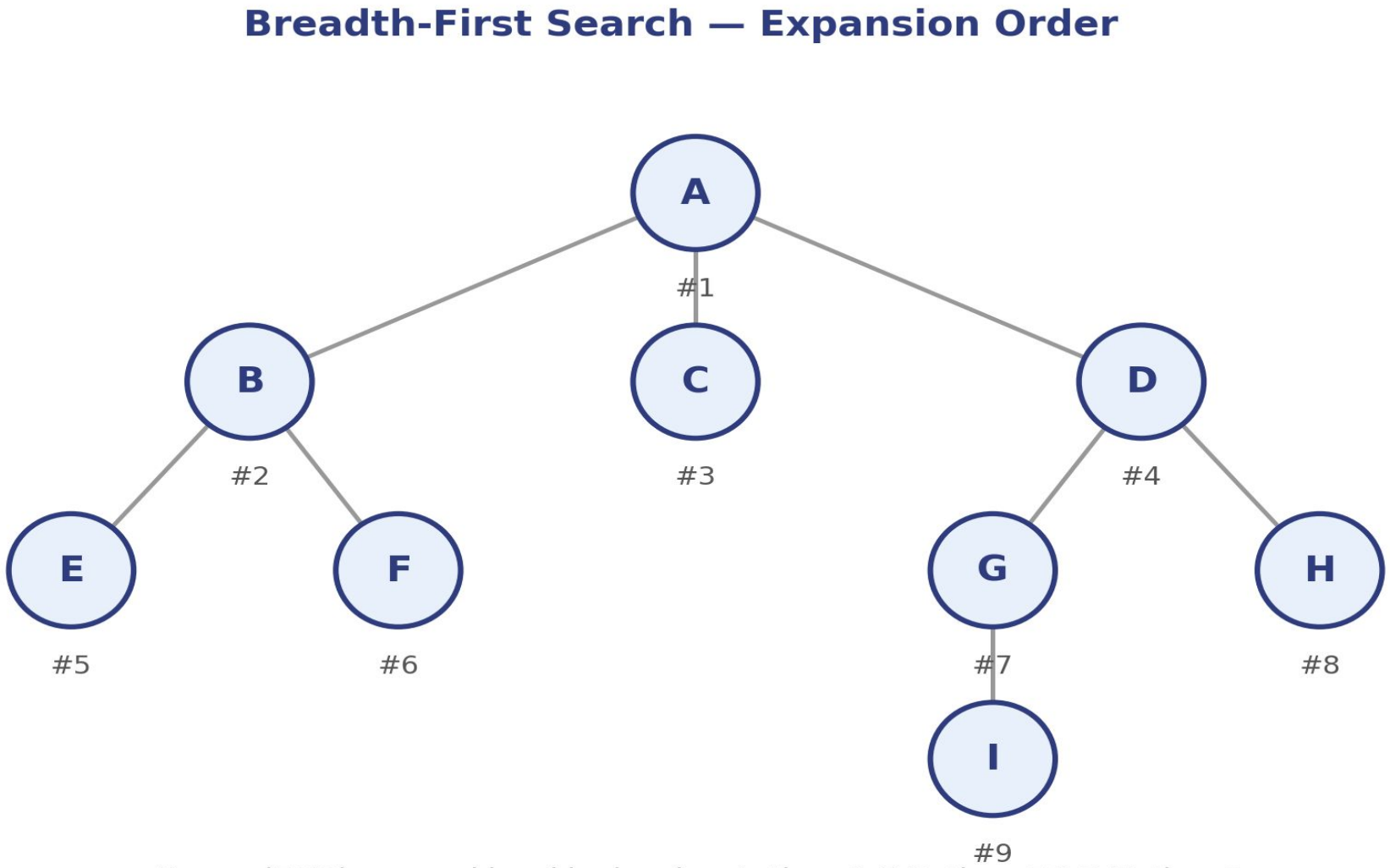


Figure 3.8 Breadth-first search on a simple binary tree. At each stage, the node to be expanded next is indicated by the triangular marker.

- **Order of expansion: A → B, C → D, E, F, G** — the triangle marks the next node to expand.
- Notice all of level 1 (B, C) is expanded before any of level 2 (D, E, F, G) — the hallmark of BFS.

BFS Simulation: A General Tree (Our Own Example)



BFS: Time and Space Complexity

- Assume every state has b successors (the branching factor) and the solution is at depth d.
- **Total nodes generated:** $1 + b + b^2 + b^3 + \ldots + b^d = O(b^d)$
- Because every node stays in memory, **space complexity is also $O(b^d)$** — usually the bigger practical problem.
- Concrete example: b=10, 1 million nodes/sec, 1 KB/node:
  - Depth d=10 ⟶ under 3 hours, but needs 10 terabytes of memory.
  - Depth d=14 ⟶ even with infinite memory, 3.5 years of runtime.
- **Lesson:** exponential-complexity search problems cannot be solved by uninformed search except for the smallest instances.

Uniform-Cost Search (Dijkstra's Algorithm)

- When actions have different costs, BFS's 'shallowest first' guarantee no longer means 'cheapest first.'
- **Uniform-cost search** (UCS) fixes this: it's best-first search with f(n) = **g(n)**, the path cost from the root.
- While BFS spreads out in waves of uniform depth, UCS spreads out in waves of uniform path-cost.
- Known in theoretical computer science as **Dijkstra's algorithm**.
- **Complexity:** $O(b^{(1+\lfloor C^*/\varepsilon \rfloor)})$, where C^* is the optimal solution cost and ε is the minimum action cost — can be much worse than b^d.
- UCS is both complete and cost-optimal — the first solution it finds is guaranteed to be cheapest.

UCS Worked Example: Sibiu to Bucharest

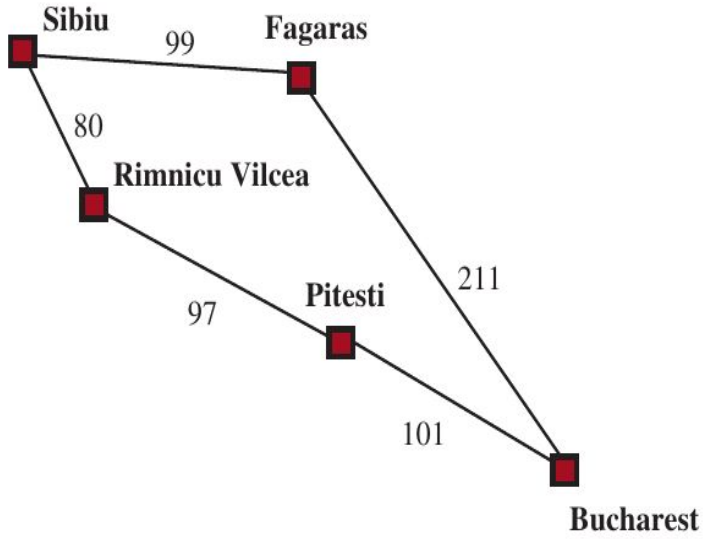


Figure 3.10 Part of the Romania state space, selected to illustrate uniform-cost search.

- **Goal:** find the cheapest path from Sibiu to Bucharest, using this subgraph of the Romania map.

UCS Worked Example: Step-by-Step Trace

Stage	What Happens	Frontier (after step)
Start	Sibiu (cost 0)	—
Step 1	Expand Sibiu → add Rimnicu Vilcea (80), Fagaras (99)	RV:80, Fag:99
Step 2	Expand Rimnicu Vilcea (cheapest) → add Pitesti (80+97=177)	Fag:99, Pit:177
Step 3	Expand Fagaras (cheapest) → add Bucharest (99+211=310)	Pit:177, Buc:310
Step 4	Expand Pitesti → 2nd path to Bucharest: 80+97+101 = 278 (cheaper than 310)	Buc:278 (was 310)
Step 5	Expand Bucharest (cost 278, now cheapest in frontier) → GOAL reached!	—

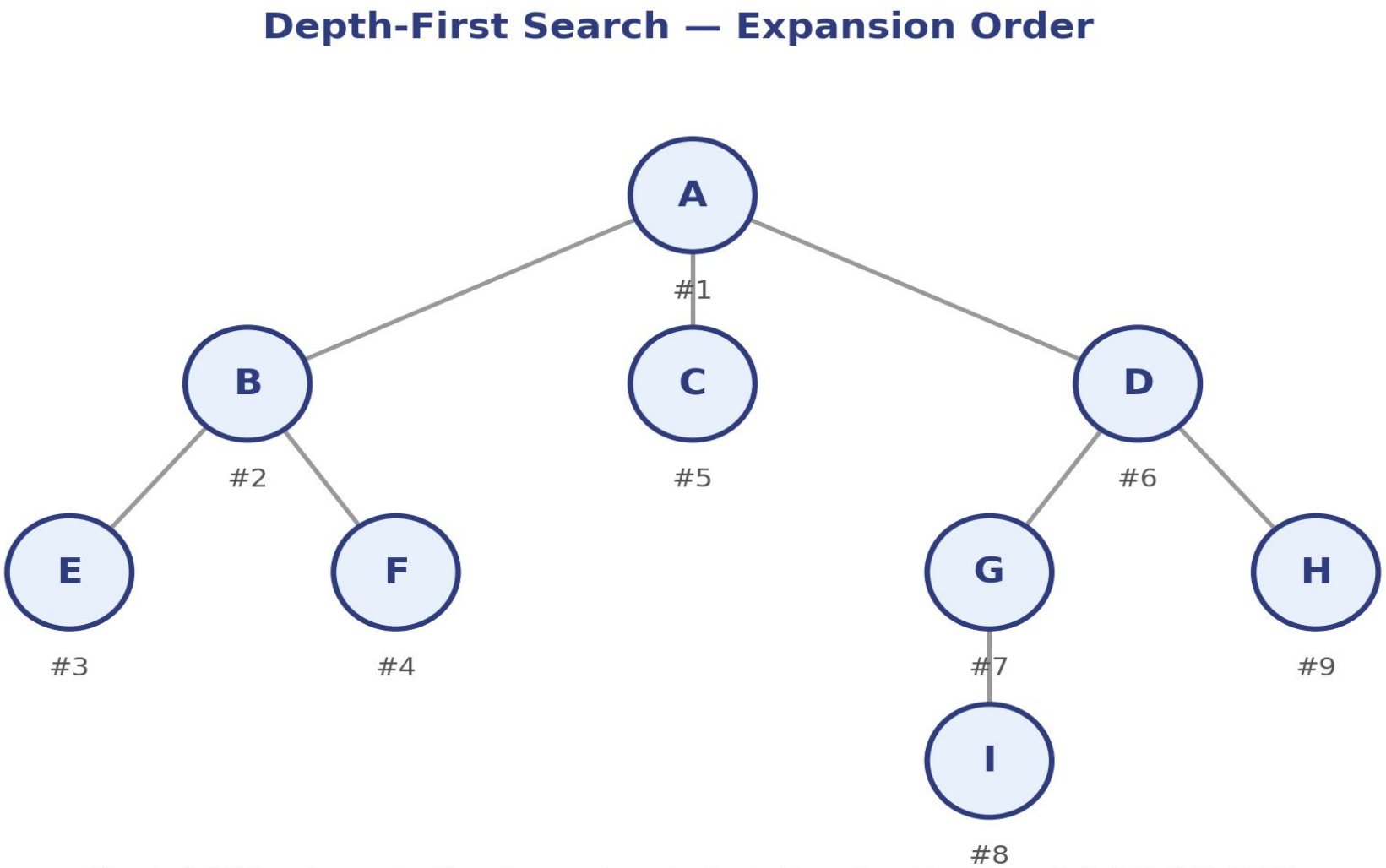
UCS: Why the Result Is Optimal

- A naive BFS would expand Fagaras before Pitesti (Fagaras is reached in **fewer hops**) and return the **Sibiu → Fagaras → Bucharest** path, costing 310 — not the cheapest!
- UCS instead always expands the **lowest path-cost** node in the frontier — so it eventually finds **Sibiu → Rimnicu Vilcea → Pitesti → Bucharest** at cost 278.
- **Key detail:** UCS tests for the goal only when a node is expanded (popped), not when generated — this is what prevents it from stopping early at a higher-cost path.
- When all action costs are equal, UCS behaves just like BFS.

Depth-First Search (DFS)

- DFS always expands the **deepest** node in the frontier first — plunging down one branch before backtracking.
- Implemented with a **LIFO stack** (Last-In-First-Out), usually as a tree-like search with no 'reached' table.
- **Not cost-optimal:** returns the first solution found, even if it isn't the cheapest.
- **Incomplete in general:** can get stuck going down an infinite (or cyclic) path, even if a solution exists elsewhere.
- **Why use it at all?** Memory: only **O(bm)** — think of BFS's frontier as the surface of an expanding sphere, and DFS's as just a single radius.

DFS Simulation: Same Tree, Different Order



Stack (LIFO): plunge to the deepest node first, then backtrack — A,B,E,F,C,D,G,I,H

DFS Simulation: Same Tree, Different Order

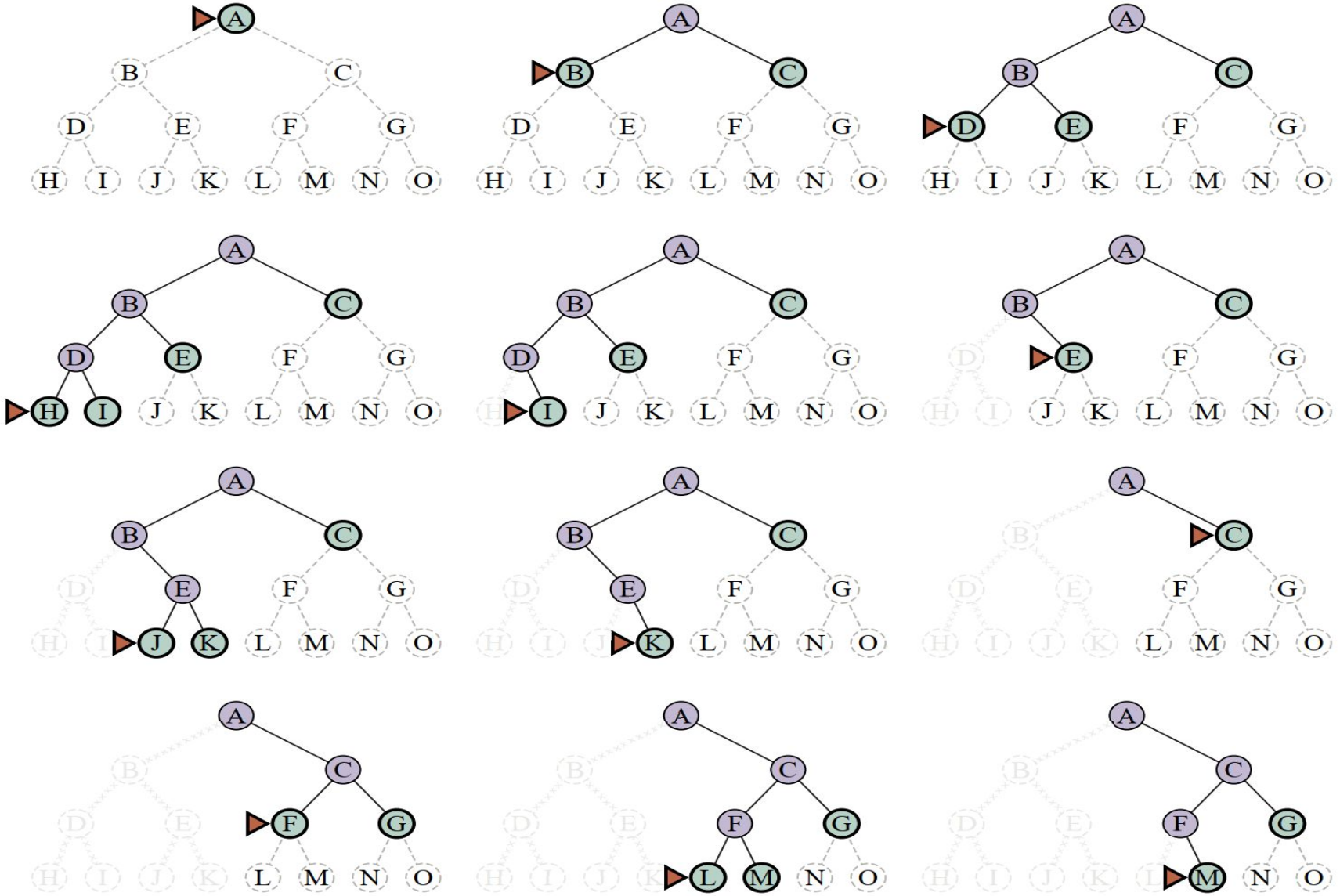


Figure 3.11: A dozen steps (left to right, top to bottom) in the progress of a depth-first search on a binary tree from start state A to goal M

BFS vs. DFS: Same Tree, Two Very Different Orders

Property	Breadth-First Search	Depth-First Search
Order of visits	A, B, C, D, E, F, G, H, I	A, B, E, F, C, D, G, I, H
Data structure	FIFO queue	LIFO stack
Memory use	O(b^d) — can be huge	O(bm) — modest
Finds shallowest goal?	Yes, guaranteed	No — may find a deep, costly one first
Complete?	Yes (if b finite)	No (infinite/cyclic spaces)

Backtracking Search: An Even Leaner DFS Variant

- **Backtracking search** generates only **one successor at a time** instead of all of them — each partially expanded node remembers which successor to try next.
- Successors are generated by **modifying the current state directly** rather than allocating memory for a brand-new state.
- This shrinks memory to just one state description plus a path of O(m) actions — a big savings over DFS's O(bm) states.
- **Requires the ability to undo** each action when backtracking.
- Critical to many large-state-description problems, such as robotic assembly and constraint satisfaction (Chapter 5).

Depth-Limited Search (DLS)

- DFS can wander down an infinite path.
- The fix: supply a **depth limit ℓ** and treat nodes at depth ℓ as if they had no successors.
- A simple modification of **tree-like DFS**.
- **Complexity:** time O(b^ℓ), space O(b^ℓ).
- **The catch:** choosing ℓ is hard.
- If ℓ < d (the solution's actual depth), DLS becomes incomplete — it may miss the solution entirely.
- If ℓ > d, DLS wastes time and is not optimal.
- Example: Romania has 20 cities, so ℓ=19 is always valid — but the graph's actual **diameter** (max shortest-path length between any two cities) is just 9, giving a much better limit.

Iterative Deepening Search (IDS)

- IDS sidesteps the 'what should ℓ be?' problem by trying **every value**: first ℓ=0, then ℓ=1, then ℓ=2, … until a solution is found.
- Combines the best of both worlds:
  - Like DFS: modest memory — O(bd) when there is a solution.
  - Like BFS: complete and optimal (when step costs are equal).
- Seems wasteful — states near the top are regenerated many times — but this barely matters, because most nodes in a tree live at the **bottom** level. The total number of nodes generated in the worst case is

$$N(\text{IDS}) = (d)b^1 + (d-1)b^2 + (d-2)b^3 \cdots + b^d$$

- **Numeric comparison (b=10, d=5):**

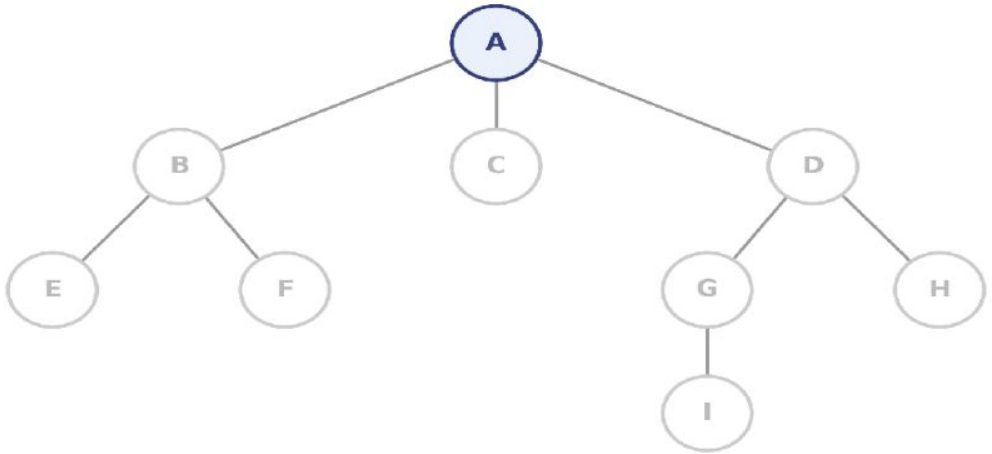
N(IDS) = 50+400+3;000+20;000+100;000 = 123,450

N(BFS) = 10+100+1;000+10;000+100;000 = 111,110

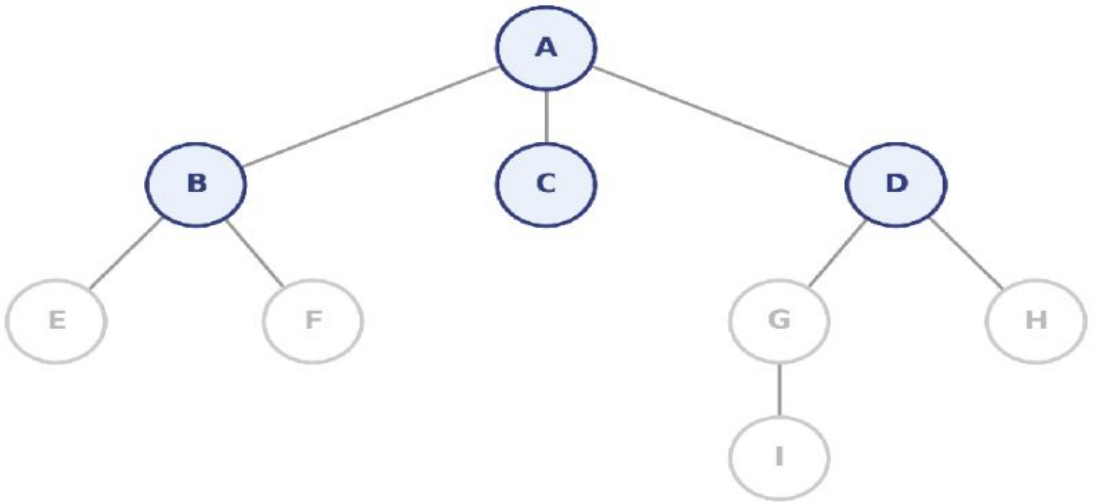
N(IDS) = 123,450 vs. N(BFS) = 111,110 — only ~11% more work for vastly less memory.

IDS Simulation: Growing the Depth Limit Step by Step

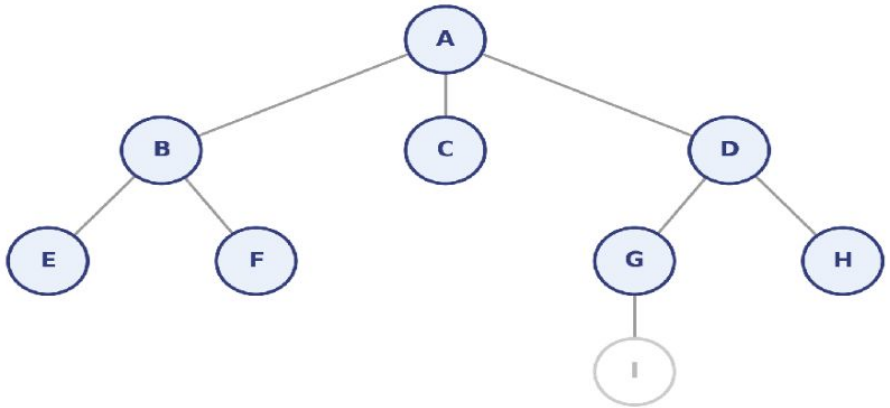
Iteration ℓ=0: only A visited



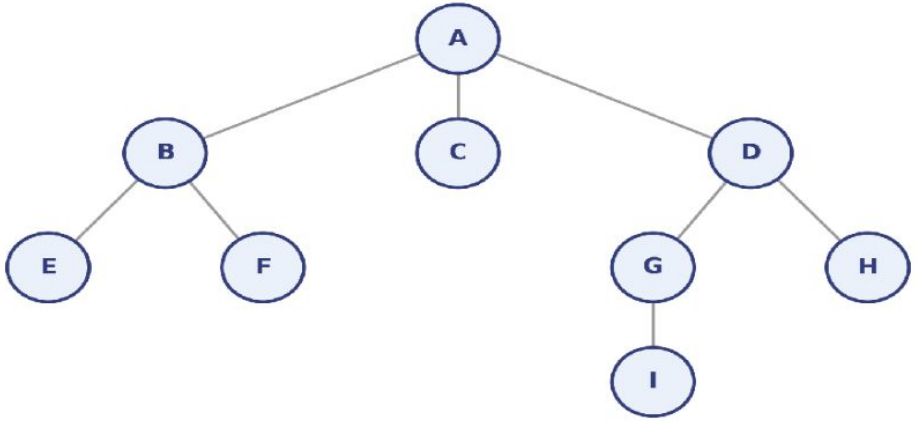
Iteration ℓ=1: A,B,C,D visited (re-visits A)



Iteration ℓ=2: adds E,F,G,H (re-visits A,B,C,D)



Iteration ℓ=3: adds I — entire tree now reached



IDS Simulation: Growing the Depth Limit Step by Step

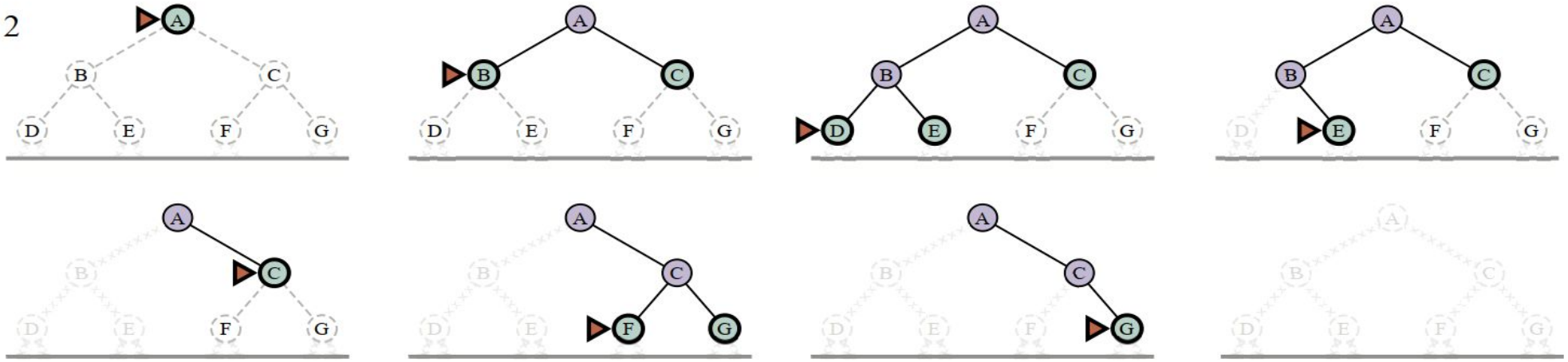
limit: 0



limit: 1



limit: 2



IDS Simulation: Growing the Depth Limit Step by Step

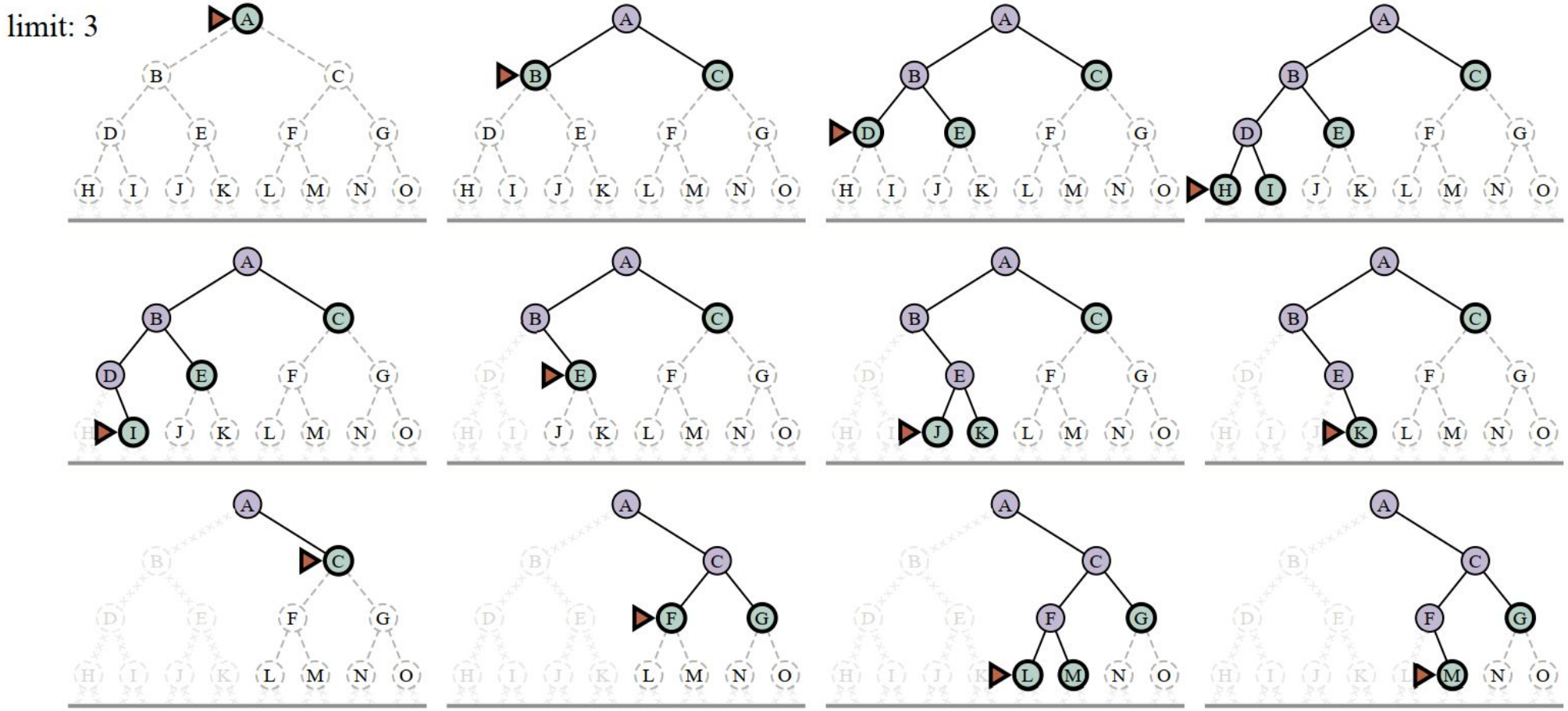


Figure 3.13 Four iterations of iterative deepening search for goal M on a binary tree, with the depth limit varying from 0 to 3.

Bidirectional Search

- **Idea:** search simultaneously forward from the initial state and backward from the goal state(s), hoping the two searches meet in the middle.
- **Motivation:** b^(d/2) + b^(d/2) is far smaller than b^d — e.g., 50,000× less work when b = d = 10.
- Requires knowing the goal state(s) in advance, and a way to compute predecessors (reverse actions).
- Often implemented as two breadth-first searches; when both use path cost as f(n), we get bidirectional uniform-cost search.
- **Challenges:** detecting when the frontiers meet, and the memory cost of maintaining two search trees at once.

Comparing Uninformed Search Algorithms

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes[1]	Yes[1,2]	No	No	Yes[1]	Yes[1,4]
Optimal cost?	Yes[3]	Yes	No	No	Yes[3]	Yes[3,4]
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$

Figure 3.15 Evaluation of search algorithms. b is the branching factor; m is the maximum depth of the search tree; d is the depth of the shallowest solution, or is m when there is no solution; ℓ is the depth limit. Superscript caveats are as follows: [1] complete if b is finite, and the state space either has a solution or is finite. [2] complete if all action costs are $\geq \epsilon > 0$; [3] cost-optimal if action costs are all identical; [4] if both directions are breadth-first or uniform-cost.

Real-World Applications of Uninformed Search

Network Pathfinding

Establishing routes for data packets between computers across a network.

Web Crawling

Crawlers follow links recursively to index pages for search engines.

Social Network Connections

Discovering friend-of-a-friend and other indirect relationships to suggest new connections.

Summary

- A problem-solving agent follows four phases: **goal formulation → problem formulation → search → execution**.
- A search problem is defined by its state space, initial state, goal states, actions, transition model, and action-cost function.
- Best-first search unifies BFS, UCS, and DFS as one algorithm with different evaluation functions f(n).
- **BFS** is optimal for uniform costs but memory-hungry; **UCS** is optimal for any positive costs; **DFS** is memory-efficient but not complete or optimal.
- **DLS** and **IDS** fix DFS's incompleteness, with IDS combining BFS's guarantees and DFS's low memory use.
- **Bidirectional search** can cut the search space dramatically when the goal is known.

Next Lecture

Chapter 3 (continued): Informed (Heuristic) Search Strategies

Greedy best-first search, A*, and heuristic design

Questions?